

Project # 1 : Credit Card Fraud Detection

Baudart Alexandre

Last update : August 29, 2026

Contents

1	Dataset and Exploratory Analysis	3
1.1	Dataset Overview	3
1.2	Features Description	4
1.3	Missing Values	4
1.4	Correlation Analysis	4
2	Preprocessing and Modeling	6
2.1	Data Splitting and Preprocessing	6
2.2	Models	6
2.3	Hyperparameter Optimization	7
2.4	Training	9
2.5	Evaluation	9
3	Results and Discussion	10
3.1	Classical Machine Learning Models	10
3.2	Multi-Layer Perceptron Experiments	12
3.2.1	Validation Strategy	12
3.2.2	Imbalance Handling Experiments	12
3.2.3	Activation Function Experiments	13
3.2.4	Architectural Ablation Studies	13
3.2.5	Dropout Experiments	14
3.2.6	Final MLP Performance	15
3.3	Confusion Matrix Analysis	16
3.4	Visual Performance Analysis	16
3.5	Overall Discussion	17
4	Conclusion	18

1 Dataset and Exploratory Analysis

1.1 Dataset Overview

The dataset **Credit Card Faud Detection** [3] has been used for this project. It was created in the context of the research works of *Dal Pozzolo et al.* [2, 1] over the extreme class unbalancing.

This dataset is about credit card fraud detection. It includes anonymized transactions made by European credit card holders.

The dataset contains 284,807 transactions going along with a binary label :

- class 0 : legitimate transaction
- class 1 : fraudulent transaction

Nevertheless, this dataset is **extremely unbalanced**. Indeed, the fraudulent transactions represent only 0.18% of the whole dataset (492 examples) against 284,315 examples of legitimate transactions.

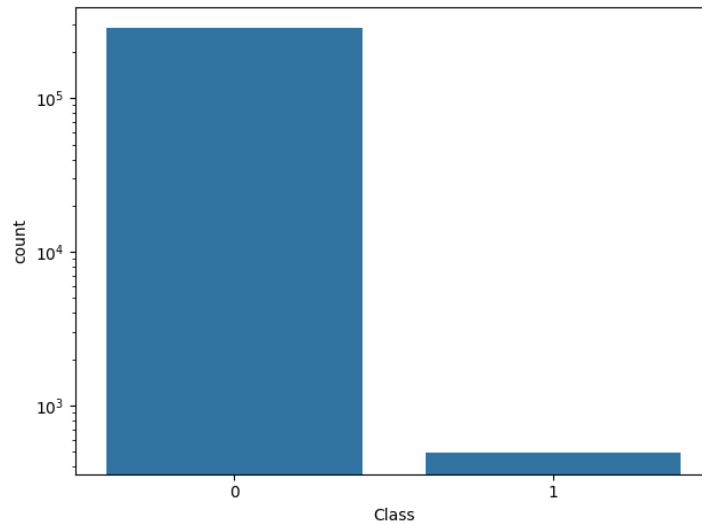


Figure 1: Countplot of the target variable *Class* (logarithm scale)

This tremendous unbalancing was one of the main challenges of the project and justify the using of adapted metrics like PR-AUC.

1.2 Features Description

The dataset includes 30 explanatory variables (features) and 1 binary target variable (label).

Among those features :

- *Time* represents the number of seconds elapsed between this transaction and the first one in the dataset.
- *Amount* is the amount of the transaction.

Unfortunately, due to confidentiality issues, the 28 others original features are not provided. Instead, these 28 features (*V1* to *V28*) are components transformed by PCA. Consequently, we don't have a lot of background information about the data, making the interpretation of the results quite tough.

1.3 Missing Values

There is not any missing value in this dataset, that simplified the preprocessing.

1.4 Correlation Analysis

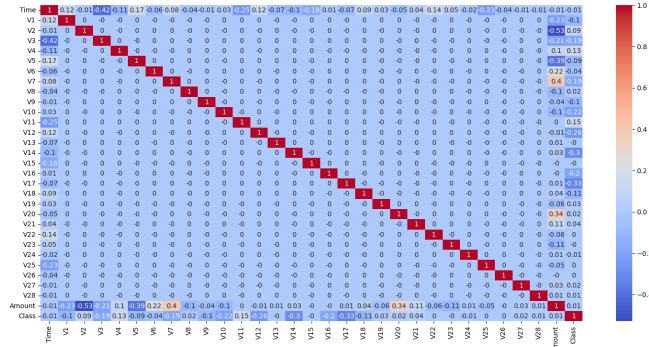


Figure 2: Correlation analysis including the target variable (*Class*)

A correlation analysis has been produced in order to identify the most related variables to the target variable.

Results show that any individual variable is not significantly correlated with the target. This suggests that fraud detection could be based on complex interactions between several variables rather than one dominant feature.

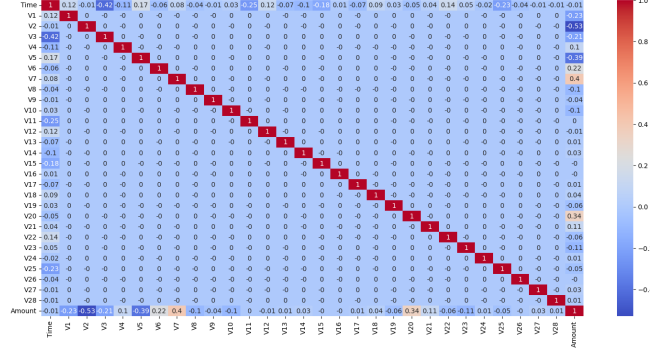


Figure 3: Correlation analysis excluding the target variable

An other analysis, excluding the target variable, shows that there are some moderate correlations between the features :

- V2 presents a negative correlation about -0.53 with *Amount*.
- V7 shows a positive moderate correlation around 0.40 with *Amount*.

However, the reading of those relations remains limited because the PCA components does not have an explicit business meaning.

2 Preprocessing and Modeling

2.1 Data Splitting and Preprocessing

The dataset was split into training and test sets using an 80/20 stratified split in order to preserve the original class distribution across all subsets. Given the extreme class imbalance of the dataset, stratification was essential to ensure representative distributions of both classes in each split.

All classical machine learning models were evaluated on the train dataset using stratified k-fold cross-validation in order to ensure robust performance estimation and stable variance across folds.

For the MLP model, a single stratified train/validation split was used (20% of the training set), generated via `StratifiedShuffleSplit`, and preserved throughout training. The validation set was used for monitoring performance during training, early stopping, and learning rate scheduling. Moreover, using a k-fold cross-validation would have been impractical due to the high computational cost of the neural network training and the cross-validation as well.

Furthermore, feature preprocessing was applied depending on the model type :

- Standardization was applied for Logistic Regression, SVM and the MLP model.
- Tree-based models (Random Forest and XGBoost) were trained on raw features without normalization, as they are invariant to monotonic transformations.

2.2 Models

The following models were evaluated in this study :

- Logistic Regression
- Random Forest
- XGBoost
- Multi-Layer Perceptron (MLP)

The selection covers a wide range of model families, from linear models to ensemble methods and deep learning architectures.

2.3 Hyperparameter Optimization

Hyperparameter tuning was performed using :

- Optuna for classical machine learning models.
- Ray Tune for the MLP model.

For computational efficiency, **30% of the training set** was used for hyperparameter optimization.

The best configurations obtained are summarized below.

LOGISTIC REGRESSION

Parameter	Value
C	0.008
tol	1e-3
max_iter	1000
fit_intercept	True

Table 1: Main parameters for logistic regression

RANDOM FOREST

Parameter	Value
n_estimators	366
max_depth	8
min_samples_split	2
min_samples_leaf	2
max_samples	0.54
oob_score	True

Table 2: Main parameters for random forest

XGBoost

Parameter	Value
n_estimators	2000
learning_rate	0.06
max_depth	5
min_child_weight	1
subsample	0.99
colsample_bytree	0.97
reg_alpha	0.34
reg_lambda	1.84

Table 3: Main parameters for XGBoost

Training configuration :

- early stopping based on PC-AUC evolution (rounds = 50)

MULTI-LAYER PERCEPTRON (MLP)

Parameter	Value
n_layers	2
hidden_size	64
dropout	0.18
initial learning rate	1e-3

Table 4: Main parameters for MLP

Training configuration :

- batch size = 64,
- loss function = BCEWithLogitsLoss,
- optimizer = AdamW,
- gradient clipping enabled,
- learning rate scheduling using, ReduceLROnPlateau
- early stopping based on validation loss.

CROSS-VALIDATION

For classical machine learning models, **stratified k-fold cross-validation** was performed for each trial during the hyperparameter optimization process. The **mean PR-AUC** across folds was used as the objective for hyperparameter selection, while the **standard deviation** was also recorded to assess performance stability across folds. Therefore, no additional cross-validation was required after optimization. The reported PR-AUC corresponds to the mean and standard deviation obtained across folds for the best-performing trial.

Note: Cross-validation was not used for the MLP, mainly due to its substantially higher computational cost. Instead, a dedicated validation set was created using a stratified split and used throughout training to monitor performance, apply early stopping, and adjust the learning rate through scheduling.

2.4 Training

Once a suitable set of hyperparameters was found, each model was trained using the selected configuration. For all models, PR-AUC was monitored as the primary performance metric.

PR-AUC metric was chosen because it is particularly suitable for this highly imbalanced dataset. Unlike accuracy, it focuses on the trade-off between precision and recall for the positive class, which corresponds to fraudulent transactions in this study. Therefore, it provides a more informative assessment of the model's ability to identify fraudulent transactions while limiting false positives.

2.5 Evaluation

Final model performance was evaluated on a held-out test set using the following metrics :

- Precision
- Recall
- F1-score
- ROC-AUC
- PR-AUC

A confusion matrix was also generated for each model in order to provide a detailed view of the classification results, particularly the numbers of true positives, true negatives, false positives, and false negatives.

3 Results and Discussion

3.1 Classical Machine Learning Models

The performance obtained by the classical machine learning models during the cross-validation process is summarized below. The mean PR-AUC represents the average performance across the folds, while the standard deviation indicates the variability between folds.

Model	Mean CV PR-AUC	Std CV PR-AUC
Logistic Regression	0.84	0.07
Random Forest	0.84	0.08
XGBoost	0.84	0.01

Table 5: Cross-validation results for classical machine learning models during optimization

The three models achieved a similar mean PR-AUC of approximately 0.84 during hyperparameter optimization. However, their variability across folds differed considerably. XGBoost obtained the lowest standard deviation, suggesting more consistent performance across the cross-validation folds.

The final performances on the held-out test set are presented here:

Model	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Logistic Regression	0.83	0.66	0.74	0.96	0.73
Random Forest	0.95	0.78	0.86	0.97	0.86
XGBoost	0.94	0.79	0.86	0.97	0.87

Table 6: Final performance of classical machine learning models on the test set

Logistic Regression achieved a relatively high precision of 0.83, indicating that most transactions classified as fraudulent were indeed fraudulent. However, its recall of 0.66 shows that a substantial proportion of fraudulent transactions were not detected. This resulted in lower F1-score and PR-AUC values compared with the tree-based models.

Although Logistic Regression achieved a mean PR-AUC of 0.84 during cross-validation, its standard deviation of 0.07 indicates greater variability between folds. The difference between its cross-validation and test performance also illustrates that the cross-validation score obtained during optimization should not be interpreted as a direct estimate of the final test performance.

Random Forest and XGBoost achieved substantially stronger results on the held-out test set. Both models obtained high precision and recall while maintaining strong PR-AUC values. XGBoost achieved the highest test PR-AUC of 0.87 and the highest recall of 0.80, while Random Forest achieved a slightly higher precision of 0.95.

Overall, XGBoost provided the best test performance among the classical machine learning models, while also exhibiting the lowest cross-validation variability.

3.2 Multi-Layer Perceptron Experiments

Several experiments were conducted to evaluate the impact of:

- validation strategies,
- class imbalance handling techniques,
- architectural components,
- regularization,
- and activation functions.

The baseline configuration used ReLU activation, a dropout rate of 0.18, and BCEWithLogitsLoss.

3.2.1 Validation Strategy

The use of `StratifiedShuffleSplit` was investigated to preserve the original class distribution when creating the validation set. This resulted in stable training behavior, with validation PR-AUC generally ranging between 0.75 and 0.81.

For comparison, a standard random validation split was also evaluated. Although the resulting test PR-AUC was slightly higher without stratification (0.8180 compared with 0.8150), the training process exhibited lower validation PR-AUC values, generally ranging between 0.70 and 0.75.

Therefore, `StratifiedShuffleSplit` was retained for the final configuration. In the context of this highly imbalanced dataset, maintaining a representative proportion of fraudulent transactions in the validation set provides a more reliable basis for monitoring model performance and applying early stopping.

3.2.2 Imbalance Handling Experiments

Additional experiments investigated the use of `pos_weight` to explicitly account for the severe class imbalance.

The use of `pos_weight` did not improve the model’s performance. The resulting test PR-AUC was 0.7927, compared with 0.8150 for the baseline configuration. Training remained relatively stable, but the use of class weighting did not provide a meaningful benefit in this setting.

Therefore, `pos_weight` was not retained in the final MLP configuration. These results suggest that, for this particular architecture and dataset, standard binary cross-entropy was sufficient to learn the minority class without explicit class reweighting.

3.2.3 Activation Function Experiments

The effect of the activation function was also investigated by replacing ReLU with LeakyReLU and GELU.

Both alternatives produced stable training, but neither provided a consistent improvement over ReLU. LeakyReLU converged more rapidly, while GELU required more training epochs and a longer training time without achieving a substantially higher PR-AUC.

Consequently, ReLU was retained for the final architecture due to its simple behavior, stable optimization, and competitive performance.

3.2.4 Architectural Ablation Studies

Several architectural components were removed individually to assess their contribution to model performance.

SKIP CONNECTIONS

Removing skip connections had only a limited impact on performance. The resulting model remained stable during training and achieved a test PR-AUC of 0.8353, compared with 0.8150 for the baseline configuration. This suggests that skip connections were not essential for this relatively shallow architecture. Moreover, removing them reduced the training time, providing a potential computational advantage without negatively affecting performance in this experiment.

BATCH NORMALIZATION

Removing the initial batch normalization layer had a major negative impact on performance. Validation PR-AUC remained around 0.20, and the final model achieved a test PR-AUC of only 0.2687.

This result indicates that the initial batch normalization layer played a particularly important role in the optimization of the proposed architecture. Within the experimental setting considered here, removing this component led to a substantial degradation in generalization performance.

LAYER NORMALIZATION

Removing layer normalization preserved relatively stable training but slightly reduced performance. The resulting model achieved a test PR-AUC of 0.7903, compared with 0.8353 for the selected configuration.

This suggests that layer normalization contributed positively to the final performance, although its effect was substantially less critical than that of the initial batch normalization layer.

3.2.5 Dropout Experiments

Several dropout rates were investigated, including 0.0, 0.08, 0.18, and 0.28. The configuration without dropout achieved the highest observed test PR-AUC of 0.8393. Dropout rates of 0.08 and 0.18 achieved PR-AUC values of 0.8318 and 0.8353, respectively, while increasing the dropout rate to 0.28 resulted in a lower PR-AUC of 0.8159.

Although the configuration without dropout achieved the highest observed performance, the differences between the configurations with moderate dropout remained relatively small. A dropout rate of 0.18 was therefore retained in the final configuration as a moderate regularization mechanism.

Overall, the experiments suggest that dropout had a limited impact on performance for this architecture and dataset, while excessive dropout may negatively affect the model’s ability to learn the underlying patterns.

3.2.6 Final MLP Performance

The final MLP configuration was selected based on the preceding experiments. It used ReLU activation, a dropout rate of 0.18, BCEWithLogitsLoss, StratifiedShuffleSplit, skip connections removed, and both batch normalization and layer normalization retained.

The final model achieved the following results:

Metric	Value
Train PR-AUC	0.87
Validation PR-AUC	0.81
Precision	0.77
Recall	0.82
F1-Score	0.79
ROC-AUC	0.97
Test PR-AUC	0.83

Table 7: Final MLP performance

The final MLP achieved a test PR-AUC of 0.83, demonstrating strong classification performance despite the severe class imbalance. Its recall of 0.8265 indicates that the model detected a large proportion of fraudulent transactions, although this was accompanied by a lower precision of 0.77.

Overall, the experiments show that MLP performance was particularly sensitive to normalization and validation strategy, whereas skip connections and activation functions had a comparatively limited impact. The final configuration achieved strong performance without explicit class reweighting through.

3.3 Confusion Matrix Analysis

The confusion matrices obtained on the test set provide additional insight into the behavior of each model under severe class imbalance.

Model	TN	FP	FN	TP
Logistic Regression	56851	13	33	63
Random Forest	56860	4	21	77
XGBoost	56860	4	20	78
MLP	56840	24	17	81

Table 8: Confusion matrices results

The confusion matrices obtained on the test set provide additional insight into model behavior under severe class imbalance.

Logistic Regression produced 33 false negatives, indicating that a substantial number of fraudulent transactions remained undetected. Random Forest and XGBoost substantially reduced this number while maintaining very low false-positive rates. Random Forest correctly identified 77 of the 98 fraudulent transactions, with 21 false negatives and 4 false positives. XGBoost performed slightly better, with 78 true positives, 20 false negatives, and only 4 false positives.

The MLP achieved the highest number of true positives (81) and therefore the lowest number of false negatives (17). However, it also produced more false positives (24), resulting in higher recall but lower precision than XGBoost.

Overall, the confusion matrices highlight the trade-off between detecting fraudulent transactions and limiting false alarms. XGBoost provided the best balance among the evaluated models, while the MLP achieved the highest fraud detection rate at the cost of more false positives.

3.4 Visual Performance Analysis

Since XGBoost achieved the best global performance, ROC and Precision-Recall curves are presented below in order to further illustrate its classification behavior under severe class imbalance.

Due to the highly imbalanced nature of the dataset, Precision-Recall curves provide more informative insights than ROC curves regarding fraud detection capability.

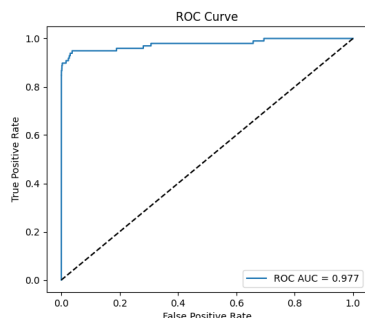


Figure 4: XGBoost ROC-AUC Curve

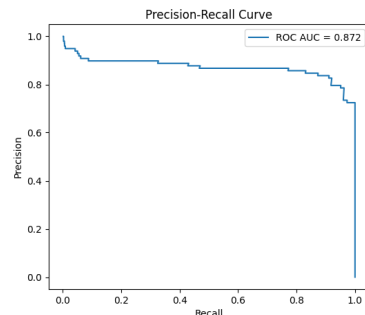


Figure 5: XGBoost PR-AUC Curve

3.5 Overall Discussion

The results show that the tree-based ensemble methods provided the strongest overall performance on the held-out test set. Logistic Regression achieved a test PR-AUC of 0.73, compared with 0.86 for Random Forest and 0.87 for XGBoost. The MLP achieved an intermediate PR-AUC of 0.83.

These results suggest that the fraud detection task involves patterns that are not fully captured by a linear decision boundary. The substantially stronger performance of the tree-based ensemble methods indicates that non-linear relationships between the available features can be exploited effectively for this task.

Among the classical models, XGBoost achieved the highest test PR-AUC and recall, while also exhibiting the lowest cross-validation variability during hyperparameter optimization. It therefore provided the strongest overall performance among the classical approaches.

The MLP also demonstrated competitive performance, particularly in terms of recall. Its recall of 0.8265 was slightly higher than that of XGBoost (0.80), meaning that it detected a slightly larger proportion of fraudulent transactions. However, this came at the cost of lower precision and a slightly lower PR-AUC.

Overall, XGBoost provided the best balance between precision, recall, and PR-AUC while requiring fewer architectural considerations than the MLP. Nevertheless, the MLP results demonstrate that neural networks can achieve competitive performance on this highly imbalanced tabular fraud detection problem when appropriate normalization, validation, and regularization strategies are employed.

4 Conclusion

The experiments conducted throughout this study highlight several important observations.

First, linear models were less effective at capturing the complex relationships underlying fraudulent transactions. Logistic Regression achieved substantially lower recall and PR-AUC on the test set compared with the ensemble and neural network approaches, suggesting that non-linear modeling capabilities are beneficial for this fraud detection task.

Second, tree-based ensemble methods proved highly effective for this problem. Random Forest and XGBoost achieved strong precision-recall trade-offs, with XGBoost obtaining the highest test PR-AUC among the classical machine learning models (0.87) and the most stable cross-validation behavior.

Third, the MLP experiments demonstrated that model architecture and training configuration have a significant impact on performance. In particular, the validation strategy and normalization layers played an important role in obtaining stable training and good generalization. The final MLP achieved a test PR-AUC of 0.8353 and a recall of 0.8265, demonstrating that neural networks can achieve competitive performance on highly imbalanced tabular data.

Overall, XGBoost achieved the strongest overall performance among the evaluated models, providing the best balance between precision, recall, PR-AUC, and cross-validation stability. The MLP nevertheless achieved highly competitive results, particularly in terms of recall, while requiring a more extensive training and architectural investigation. These results highlight the effectiveness of carefully tuned non-linear models for fraud detection under severe class imbalance.

References

- [1] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. Credit card fraud detection: a realistic modeling and a novel learning strategy. IEEE Transactions on Neural Networks and Learning Systems, 29(8):3784–3797, 2018.
- [2] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating probability with undersampling for unbalanced classification. In IEEE Symposium on Computational Intelligence and Data Mining (CIDM). IEEE, 2015.
- [3] Worldline and Machine Learning Group de l’ULB. Credit Card Fraud Detection Dataset. Dépôt Kaggle, 2016. Disponible sur : <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.